ITDPA2-44 Project

Site: Eduvos LMS
Course: Data Structures and Algorithms in Python Assessments
Book: ITDPA2-44 Project
Printed by: Musongela Nyembo
Date: Tuesday, 3 November 2026, 1:43 PM

Table of contents

1. Assignment / Project
2. Instructions to Students
3. Section A
4. Scenario
    4.1. Question 1
    4.2. Question 2
    4.3. Question 3
    4.4. Question 4

Project

Faculty:	Information Technology
Module Code:	ITDPA2-44
Module Name:	Data Structures and Algorithms in Python
Content Writer:	Mr Kasongo Kalubo
Internal Moderation:	Community of Practice

Copy Editor:	Nicole Mills
Total Marks:	100
Submission Week:	Week 6

This module is presented on NQF level 6.

5% will be deducted from the student's project mark for each calendar day the project is submitted late, up to a maximum of three calendar days. The penalty will be based on the official campus submission date.

Projects submitted later than three calendar days after the deadline or not submitted will get 0%. [1]

This is an individual project.

This project contributes 40% towards the final mark.

[1] Under no circumstances will projects be accepted for marking after the projects of other students have been marked and returned to the students.

Instructions to Students

1. Please ensure that you use the Assignment Template when preparing your submission (where applicable).
2. Please ensure that your answer file (where applicable) is named as follows before submission: **Module Code – Assessment Type – Campus Name – Student Number.**
3. Remember to keep a copy of all submitted assignments.
4. All work must be typed.
5. Please note that you will be evaluated on your writing skills in all your assignments.
6. All work must be submitted through Turnitin. The full originality report will be automatically generated and available for the lecturer to assess. Negative marking will be applied if you are found guilty of plagiarism, poor writing skills, or if you have applied incorrect or insufficient referencing. (See the 'instructions to students' book activity before this activity where the application of negative marking is explained.)
7. You are not allowed to offer your work for sale or to purchase the work of other students. This includes the use of professional assignment writers and websites, such as Essay Box. You are also not allowed to make use of artificial intelligence tools, such as ChatGPT, to create content and submit it as your own work. If this should happen, Eduvos reserves the right not to accept future submissions from you.

Section A

Learning Objective

On completion of this project, students should be able to interpret and design appropriate data structures for a defined problem, implement and test those structures in Python (including string processing and error handling), explain and apply recursive and search/sort algorithm concepts, and evaluate and justify the selection of data structures/algorithms for a real-

world application area.

Project Topic

- Emergency Incident Intake and Vehicle Lookup System using Priority Queues, Hash Tables, and String Processing
- Road Network Route-Finding and Suburb Reporting System using Graph Traversal, Recursive Algorithms, and Tree/Hash Indexing

Scope

The scope of this project is based on the topics in the module guide, starting from fundamental list-based structures through to stacks, queues, priority queues, linked lists, hashing, trees, and graphs, together with recursion and string processing.

Technical Aspects

- Python 3 is the required implementation language for all code sub-questions
- Students may use Python's standard library (e.g. collections.deque, heapq, dict) where a sub question does not explicitly require a structure to be built from first principles; where a sub-question asks students to 'implement' or 'design' a structure, the underlying logic (nodes, pointers, comparisons) must be the student's own work.
- All complexity/performance claims must be expressed using Big-O notation and justified, not merely asserted.

Marking Criteria

Sub-question	What Is Being Assessed	Marks
1.1(a)	Time-complexity analysis of the relevant list operations: append, linear search and insertion at an arbitrary position	4
1.1(b)	Explanation of why the identified limitations matter specifically in an emergency-dispatch context	4
1.2	Selection of appropriate replacement structures for R1 (incident intake) and R2 (vehicle lookup)	2
1.2(a)	Description of valid operations, justification against the current list-based approach, and research-supported justification for the selected structures	10
	Question 1 Subtotal	**20**
2.1	Implementation of a priority-based incident-intake structure that correctly handles critical and routine incidents while maintaining appropriate FIFO behaviour	8
2.2	Implementation of a vehicle and crew lookup structure that retrieves vehicle records using a registration/fleet number	5
2.3	Appropriate handling and demonstration of realistic failure conditions, including empty dispatch, missing vehicle records and invalid/missing severity	5
2.4	Testing of the implementation using distinct normal, edge and/or failure test cases with expected and actual results	4

Sub-question	What Is Being Assessed	Marks
2.5	Empirical performance comparison of vehicle lookup against linear search at different dataset sizes, including interpretation of results and research support	8
	Question 2 Subtotal	**30**
3.1	Explanation and application of recursive depth-first graph traversal, including base case, recursive case, prevention of repeated visits, and identification of a SafeRoute situation where recursion is unsuitable	10
3.2	Comparison of hash-table and linear-search lookup using Big-O notation at 50,000 vehicle records, including time/space considerations, memory trade-offs and supporting research	15
3.3	Evaluation of A* as an alternative to Dijkstra's algorithm, including the role of a heuristic and a supported suitability judgement for Ridgeview's road network	5
	Question 3 Subtotal	**30**
4.1	Working Python implementation of both a Hash Table and Binary Search Tree for the suburb index, including insertion, lookup and evidence of execution	8
4.2	Implementation of an alphabetically ordered suburb listing and explanation of the ordering implications of the two structures	6
4.3	Extension of one structure to support an additional operational requirement, including successful demonstration and justification of the design choice	6
	Question 4 Subtotal	**20**
	TOTAL	**100**

AI Use Requirements For This Assessment

This assessment is classified as AI-1, as defined in the AI Use Category Table below.

AI Use Category Table

For this specific assessment, the following applies:

Code	Category	What This Means	Evidence Required	Disclosure Required
AI-1	**Limited AI Use**	You may use AI only for simple support tasks but all the codes must still be your own.	In some tasks, you may also be asked for evidence of your process.	You must complete and submit an **AI Disclosure Appendix** explaining how you used AI. Access the AI Disclosure Appendix Template here.

Study the scenario and complete the question(s) that follow:

Emergency Incident Intake and Vehicle Lookup System using Priority Queues, Hash Tables, and String Processing

The Ridgeview Metropolitan Council currently manages emergency-vehicle dispatch (ambulance, fire, and traffic-response units) using a single spreadsheet-based log maintained by hand. All incoming incident reports – regardless of severity – are appended to the bottom of a single Python list by a call-centre application and are processed strictly in the order they were entered. The Council has approved funding for a pilot system, SafeRoute, and has hired you as a junior software developer/analyst. You have been given the following functional requirements, gathered from Council stakeholders:

- R1 – Incident intake: Incoming calls must be logged as they arrive and processed in the order received, EXCEPT those life-threatening incidents (e.g. cardiac arrest, structure fire with trapped people) must be moved ahead of routine incidents (e.g. noise complaint, minor fender-bender) regardless of arrival order.
- R2 – Vehicle and crew lookup: Dispatchers must be able to look up a specific emergency vehicle's status by its registration/fleet number almost instantly, even once the fleet grows to tens of thousands of vehicles across the wider provincial network.
- R3 – Call transcript processing: Call-centre operators type a free-text transcript for every call. The system must be able to validate and extract structured information from this text (e.g. a vehicle registration plate mentioned by a witness, or a suburb name) before it can be used elsewhere in the system. The Council's existing developer has expressed concern that 'a list is a list – we don't need anything more complicated than what we already have'. You have been asked to produce a short technical project report that responds to this position and delivers a working proof-of-concept for parts of the system.

Source: Adhicandra, Nurhidayati and Fauzan (2024); City and County of San Francisco (2026); Wang et al. (2023).

Question 1 — 20 Marks

1.1 Critically analyse the limitations of the current list-based intake and lookup approach against Requirements R1 and R2 as the system scales to a provincial network. Your answer must:

a. Reference the time complexity of the relevant list operations (append, linear search, insertion at an arbitrary position). (4 Marks)

b. Explain, in your own words, why these limitations matter in an emergency-dispatch context specifically – not merely in general terms. (4 Marks)

[Subtotal: 8 Marks]

1.2 Select an appropriate data structure for R1 and R2. (2 Marks)

a. For each structure:

- State the structure chosen.
- Describe the valid operations that can be performed on it.
- Justify why it is more appropriate than the current list.
- Support your justification with at least one source beyond the prescribed textbook describing how a comparable real-world system addresses the same problem. (10 Marks)

[Subtotal: 12 Marks]

End of Question 1

Question 2 — 30 Marks

Study Scenario 1 above and complete the question(s) that follow, building working Python code for the structures specified in Question 1.

Study Scenario 1 and complete the questions that follow. Your solution must use appropriate Python data structures to address the SafeRoute incident-intake and vehicle-lookup requirements identified in Question 1.

2.1 SafeRoute requires critical incidents to receive priority while maintaining the order in which incidents of the same priority were received. Develop a Python solution that:

- Stores incoming incidents with an appropriate priority.
- Allows routine incidents to be processed in the order received.
- Allows critical incidents to be dispatched ahead of routine incidents.
- Demonstrates the result using at least **four incidents**, including both routine and critical incidents.

Include appropriate output to demonstrate that the incidents are dispatched in the expected order. (8 Marks)

2.2 Develop a Python vehicle and crew lookup solution that stores a vehicle record using its registration or fleet number as the lookup key. Your demonstration must add at least three vehicle records and successfully retrieve a specified vehicle's details. (5 Marks)

2.3 Improve the reliability of your solutions from Questions 2.1 and 2.2 by handling the following three situations appropriately:

- An attempt to dispatch when no incidents are waiting.
- A lookup for a vehicle registration/fleet number that does not exist.
- An attempt to add an incident with a missing or invalid priority/severity.

Demonstrate through program output that each condition is detected and handled appropriately. (5 Marks)

2.4 Test your implementation using at least four distinct test cases. Your tests must include normal operation as well as at least one edge or failure condition. For each test, state what is being tested, the expected result and whether the actual result meets the expectation. (4 Marks)

2.5 The Council wants evidence that the vehicle-lookup approach will remain suitable as SafeRoute expands. Using Python timing functionality, compare the lookup performance of your vehicle structure against a linear search through a Python list at two different dataset sizes.

Present the results in a small table or suitable program output and briefly explain:

- What happens to the lookup time as the dataset becomes larger.
- Whether your observations support the data-structure decision made in Question 1.

Support your interpretation with at least one relevant external source beyond the prescribed textbook. (8 Marks)

[Subtotal: 30 Marks]

End of Question 2

Question 3 — 30 Marks

Study the scenario and complete the question(s) that follow:

Road Network Route-Finding and Suburb Reporting System using Graph Traversal, Recursive Algorithms, and Tree/Hash Indexing

Six months into the SafeRoute pilot described in Scenario 1, the Council has approved a second phase: connecting the intake and lookup systems from Phase 1 to a live model of Ridgeview's road network and building a proper reporting layer on top of the incident data the system has been collecting. You remain the developer/analyst responsible for the pilot. The Council has provided two further sets of requirements:

- R4 – Route-finding: The road network must be modelled so that the system can determine whether a path exists between the dispatch depot and an incident location and can traverse the network to identify candidate routes.
- R5 – Historical audit trail: Auditors need to be able to browse the chronological history of incidents in a given suburb, and dispatch supervisors have asked for the ability to 'undo' the last dispatch action taken in error during a shift.
- R6 – Reporting by suburb: Monthly reports must list all incidents grouped and retrievable by suburb, and management has asked whether it would be better to keep this index sorted (so it can also produce alphabetical suburb listings) or optimised purely for lookup speed.

The Council's roads engineer has pointed out that Ridgeview's street layout is irregular, with several one-way sections and a river crossed by only two bridges, so any 'path exists' check has to work on the network as it actually is, not on a simplified grid. As with Phase 1, you are required to research at least one real-world routing, logistics, or municipal-reporting system and draw on that research where the questions below ask you to justify your design against real-world practice.

Source: Tian et al. (2025)

Question 3 – 30 Marks

3.1 Explain how a recursive depth-first search (DFS) could be used to determine whether a path exists between the SafeRoute dispatch depot and an incident location in Ridgeview's road network.

In your answer:

- Explain the base case and the recursive case.
- Explain how the algorithm avoids repeatedly visiting the same road location.
- Identify one situation elsewhere in the SafeRoute system where recursion would not be suitable, explaining why an iterative or alternative approach would be better.

(10 Marks)

3.2 The Council must decide between two approaches for locating a vehicle record:

- Option A: the hash-table-based lookup used in SafeRoute.
- Option B: linear search through an unsorted Python list.

Compare the two approaches using Big-O notation for time and space requirements.

Assume the system contains 50,000 vehicle records. Use this scale to explain the likely difference in the amount of work required for a lookup, and discuss the memory trade-off involved in using a hash table.

Support your discussion with **at least one relevant external source beyond the prescribed textbook**.

(15 Marks)

3.3 Evaluate whether the **A*** algorithm would be a suitable alternative to Dijkstra's algorithm for route-finding in Ridgeview.

In your answer:

- Explain one important difference between the two algorithms.
- Explain how a heuristic could guide A* toward the incident location.
- Give a supported judgement about which approach is more suitable for SafeRoute's road network.

(5 Marks)

[Subtotal: 30 Marks]

End of Question 3

Question 4 — 20 Marks

4.1 SafeRoute must maintain an index of suburbs for the reporting requirement described in R6. Implement both a Hash Table and a Binary Search Tree (BST) in Python for this purpose.

For each structure:

- Populate it with at least eight suburb names.
- Implement an operation for adding a suburb.

- Implement an operation for finding a specified suburb.

Provide suitable output demonstrating that both structures operate correctly. (8 Marks)

4.2 Management requires SafeRoute to produce an alphabetical list of all suburbs included in the index.

Using the structures implemented in Question 4.1:

- Implement a solution that produces the required alphabetical listing.
- Explain how the structure you selected produces the ordered output.
- Explain what additional processing would be required to produce the same result using the other structure.

Provide evidence of successful execution. (6 Marks)

4.3 The suburb index must now support one additional operational requirement. Select ONE of the following:

- Maintain the number of incidents recorded for each suburb.
- Update a suburb name.
- Remove a suburb from the index.

Extend one of your structures from Question 4.1 to support the selected requirement.

Demonstrate that your extension works and briefly justify why the structure you selected is appropriate for the requirement. (6 Marks)

[Subtotal: 20 Marks]

End of Question 4